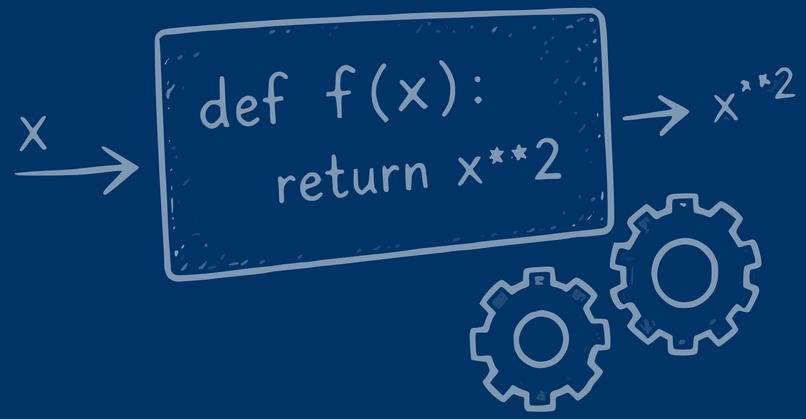


Programming

Lecture 08 — Functions

Naming a piece of work, and using it again

Dr. Ramon Winterhalder

Human-Centered Artificial Intelligence · A.Y. 2026/27

UNIVERSITÀ DEGLI STUDI DI MILANO

INFN MILANO

Where we left off

Lecture 07, in four lines

A tuple is one thing in parts
(3, 4). Unpack it with x, y = p.

A set holds distinct values
x in s is fast at any size.

A dictionary maps keys to values
d[key], and d.get(key, default) when it may be missing.

Counting in one pass
counts[x] = counts.get(x, 0) + 1

Every structure so far came with functions somebody else wrote. len, sorted, max, get. Today you write your own.

Three promises come due. key=str.lower from lecture 6. key=counts.get from lecture 7. And the cleaning line that the lecture 7 notebook wrote twice.

By the end of today you can…

- define a function and call it
- say what return does that print does not
- give parameters default values, and call with keyword arguments
- say which names a function can see, and which it cannot
- pass a list to a function without surprises
- split a program into functions, from the top down

The exam has a question that says "write a function". From today, every sheet has several.

A

Defining and calling

def · parameters · return · docstrings

You have called functions since week one

What they all have in common

the call	gives back	and on the way
`len("abc")`	`3`	
`int("42")`	`42`	
`max([3, 9, 2])`	`9`	
`round(2.567, 1)`	`2.6`	
`sorted([3, 1, 2])`	`[1, 2, 3]`	a new list
`print("hi")`	None	shows hi on the screen

A function is a named piece of work. You hand it values, it hands back one value. The brackets mean "do it now".

print is the odd one out. It does something you can see, and hands back nothing. Keep that in mind for four slides.

Why write your own?

Four reasons

Name an idea

mean(grades) says what it means. sum(grades) / len(grades), written out every time, does not.

Write it once

Lecture 7 cleaned words with the same line in two places. Change one and forget the other, and the program disagrees with itself.

Test it in pieces

A function can be checked on its own with three calls, before the rest of the program exists.

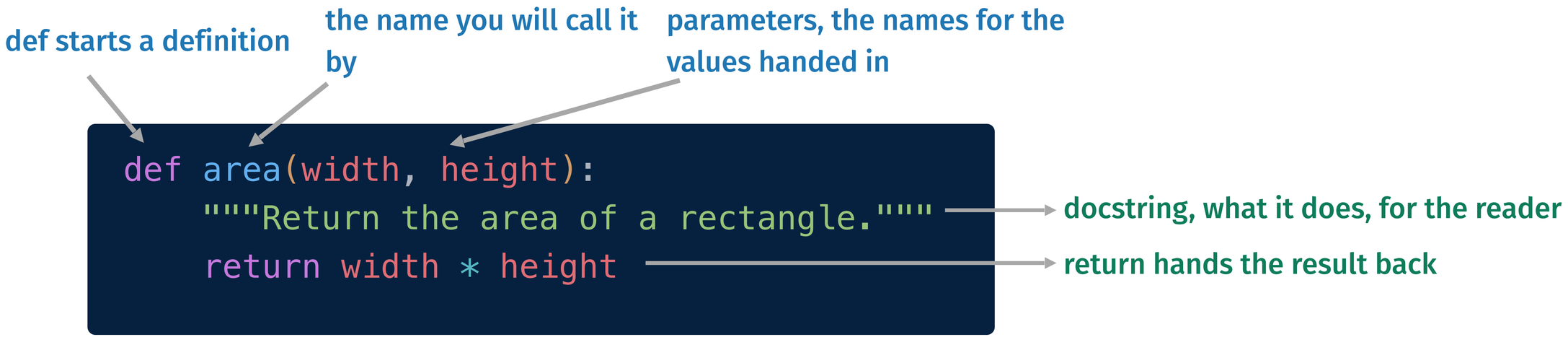
Hide the detail

You used sorted for weeks without knowing how it sorts. Your own functions can be used the same way.

The test for "should this be a function?"
Can you give it a short, honest name. If you can, it probably should be.

Anatomy of a definition

Every part has a job



The colon and the indentation work exactly as for if and for. The body is everything indented under the def.

Read it aloud as a sentence. To find the area of a width and a height, return width times height.

Defining is not running

A recipe in a book feeds nobody

Define

```
def greet():
    print("hello")
```

Nothing is printed. Python stores the recipe under the name greet and moves on.

Call

```
greet()
greet()
```

Now it runs, once per call. hello appears twice.

In a script, order matters

```
greet()                 ->  NameError: name 'greet' is not defined
def greet():                    Python has not read the definition yet
    print("hello")
```

A script runs top to bottom. Define first, call after. In a notebook the order of running cells counts, not the order on the screen.

What happens in a call

print(area(3, 4)), step by step

1	**The arguments are worked out**	`3 and 4`
2	**The parameters are attached to them**	`width = 3, height = 4`
3	**The body runs**	`width * height is 12`
4	**return hands the value back**	`the call ends here`
5	**The call is replaced by that value**	`print(area(3, 4)) is now print(12)`

Step 5 is the whole idea. A call is an expression, like `3 + 4`. Wherever you can write a value, you can write a call that gives one.

return is not print

The most common confusion in this course

Prints

```
def double_p(x):
    print(x * 2)

y = double_p(5)       # shows 10
print(y)              # None
```

```
double_p(5) + 1  ->  TypeError:
unsupported operand type(s) for +:
'NoneType' and 'int'
```

Returns

```
def double_r(x):
    return x * 2

y = double_r(5)       # shows nothing
print(y)              # 10
```

```
double_r(5) + 1  ->  11
```

print shows a value to a person. return hands a value to the program. Only a returned value can be stored, compared or calculated with.

return ends the function

And a function without return gives back None

```
def sign(x):
    if x < 0:
        return -1
    elif x == 0:
        return 0
    return 1

sign(-4), sign(0), sign(7)    ->  (-1, 0, 1)
```

- The first return that runs ends the call. Nothing after it in the function runs.
- So the last line needs no else. If x were negative or zero, we would have left already.
- Fall off the end without a return, and the call gives back None.

Two answers? Return a tuple.

```
def min_max(xs):
    return min(xs), max(xs)
lo, hi = min_max([4, 9, 1])    # lo is 1, hi is 9
```

Parameters and arguments

Matched by position

```
def power(base, exp):          # base and exp are parameters
    return base ** exp

power(2, 3)    ->  8           # 2 and 3 are arguments
power(3, 2)    ->  9
```

- Parameters are the names in the definition. Arguments are the values in the call.
- The first argument goes to the first parameter, the second to the second. Swap them and you get a different answer, with no warning.
- The count must match exactly.

```
power(2)       ->  TypeError: power() missing 1 required positional argument: 'exp'
```

Docstrings

Say what it does, not how

```
def mean(xs):
    """Return the mean of the numbers in xs.

    xs must not be empty.
    """
    return sum(xs) / len(xs)
```

- The first string in the body. Three quotes, so it can span lines.
- What goes in, what comes back, and any condition the caller must respect.
- help(mean) shows it, and so does the notebook when you press Shift + Tab inside the brackets.

If the docstring is hard to write, the function is doing too much, or nobody has decided yet what it should do. Both are worth knowing before you write the body.

"xs must not be empty" is a promise the caller has to keep. mean([]) still divides by zero, and the docstring says whose fault that is.

Worked example

From a specification, and from old code

What you have

A specification in one sentence: how many days has someone of a given age been alive?

The exam session program from lecture 6, written as one block.

What you want

A function days_alive, written from the sentence, with a docstring and three test calls.

The exam session program again, with the same output, built from small named functions and a main.

Write the docstring and the test calls first, then the body. When the tests pass, the function is done.

In the notebook, part A. Built live, starting from the specification.

Exercises — Block A

Sheet 08, part A · 40 minutes · docstring first

- Predict what four short programs print, return or both.
- Find the bug in four functions that run and give wrong answers.
- Write two functions from their descriptions.
- Replace copied code with one function.

For every function you write: the docstring, then three test calls with the answers you expect, then the body.

B

Arguments and scope

defaults · keywords · local names · lists as arguments

Default values

Parameters that can be left out

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

greet("Ada")              ->  "Hello, Ada!"
greet("Ada", "Ciao")      ->  "Ciao, Ada!"
```

You have used them all along

```
round(x)          ⟶   round(x, 2)
sorted(xs)        ⟶   sorted(xs, reverse=True)
print(a, b)       ⟶   print(a, b, sep=", ")
```

Parameters with a default come after the ones without. The other way round is a SyntaxError.

Keyword arguments

Name the parameter in the call

```
greet(greeting="Ciao", name="Ada")    ->  "Ciao, Ada!"
sorted(grades, reverse=True)
print("a", "b", sep="")
```

- A keyword argument says which parameter it is for, so its position no longer matters.
- They make calls readable. sorted(xs, reverse=True) says what True is for. sorted(xs, True) would not.
- Positional arguments first, keyword arguments after.

```
greet(name="Ada", "Ciao")   ->  SyntaxError: positional argument follows keyword argument
```

reverse=True and sep="" were keyword arguments all along. The syntax is the same one used to give defaults in the definition.

Functions are values too

The key= promise, kept

```
def surname(full):
    return full.split()[-1]

names = ["Grace Hopper", "Ada Lovelace", "Alan Turing"]

sorted(names)                  # by first name
sorted(names, key=surname)     # by surname
```

- key= takes a function. sorted calls it on every item and sorts by what it returns.
- key=surname, without brackets: hand the function over. key=surname() would call it right now, with no argument.
- str.lower, counts.get and len work the same way. They are functions with names.

Lecture 6 sorted names with key=str.lower. Lecture 7 sorted words with key=counts.get.
Both were this: a function handed to another function.

A name followed by brackets runs the function. A name on its own is the function itself, a value like any other.

Local names

What happens in a function stays in the function

```
x = 10

def f():
    x = 99          # a new, local x
    return x

print(f())          # 99
print(x)            # 10
```

the program

x → 10

one call of f, gone when it returns

x → 99

Parameters and names assigned inside a function are local. Each call gets fresh ones, and they disappear when the call ends.

This is a feature. You can name a variable total inside a function without checking whether some other part of the program uses total too.

Reading outside names, and the trap

Pass values in, return results out

Reading works

```
rate = 0.22

def tax(price):
    return price * rate

tax(100)    ->  22.0
```

Because count is assigned inside add, it is local for the whole function, including the right-hand side that runs first.

Assigning makes it local

```
count = 0

def add():
    count = count + 1

add()
```

```
UnboundLocalError: cannot access local
variable 'count' where it is not
associated with a value
```

The fix is not the global keyword, although it exists. Hand the value in as a parameter and return the new one: count = add(count).

Passing a list to a function

Lecture 6, from the inside of a function

Changes the caller's list

```
def add_bonus(xs):
    for i in range(len(xs)):
        xs[i] += 1

g = [27, 30]
add_bonus(g)
print(g)          # [28, 31]
```

Leaves it alone

```
def with_bonus(xs):
    return [x + 1 for x in xs]

g = [27, 30]
h = with_bonus(g)
print(g, h)       # [27, 30] [28, 31]
```

The parameter is a second name for the same list, exactly like b = a. A change through xs is a change to g.

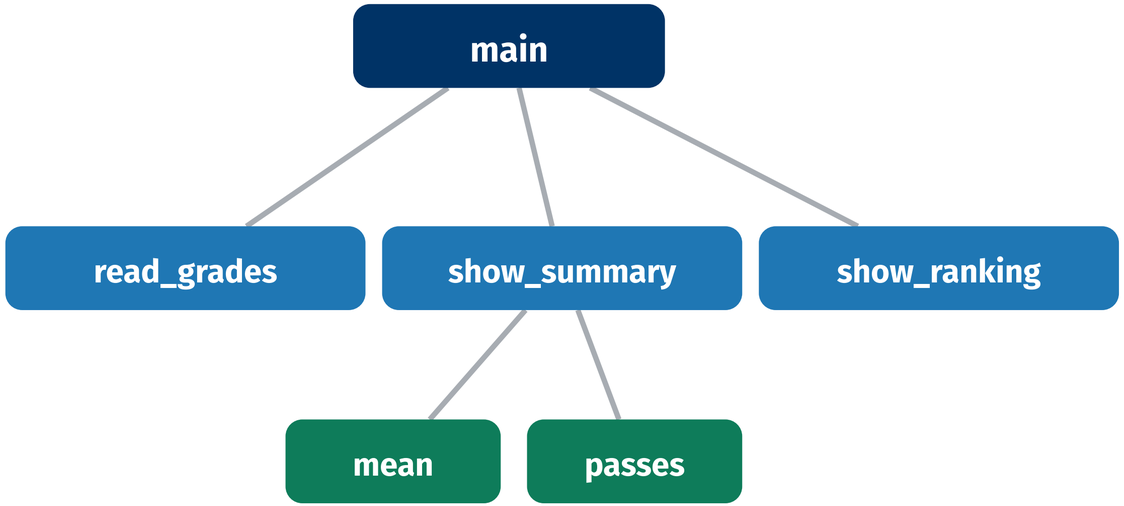
Pick one style per function. Change the list in place and return None, like .sort(). Or return a new list and leave the input alone, like sorted(). Doing both confuses everyone who calls it.

Top-down design

The comment plan grows into functions

```
def main():
    grades = read_grades()
    show_summary(grades)
    show_ranking(grades)

main()
```



- Write main first, calling functions that do not exist yet. It reads like the plan.
- Then write each function, and test it on its own.
- A function that is still too big gets the same treatment, one level down.

Your comment plan from lecture 2 was always this. Each comment line becomes the name of a function.

Worked example

A small calculator

What you have

A specification. Show a menu of four operations and q to quit. Read a choice and two numbers. Print the result. Repeat until q.

What you want

- One function per operation.
- A function that shows the menu and reads a valid choice.
- Division by zero and unknown choices handled politely.
- The whole program in a main of a few lines.

Design it top down. Write main first, with calls to functions that do not exist yet, then fill them in one by one.

In the notebook, part B. Built live, from main downwards.

Exercises — Block B

Sheet 08, part B · 40 minutes · plan with function names

Read and predict

- Seven calls with defaults and keywords, three of them broken.
- Which names a function can see.
- Four functions, and whose list they change.

Write

- Remove the failed grades, twice: in place and as a new list.
- A weather report, designed top down.

Before writing a function, decide which of the two styles it has. Does it change what it is given, or hand back something new?

Today in one slide

def name(parameters):

A named piece of work. Defining stores it, calling runs it.

return, not print

return hands a value to the program. Without it, the call gives None.

Defaults and keywords

greet(name, greeting="Hello"). Call with greeting="Ciao".

Functions are values

key=surname hands the function over. Brackets would call it.

Names inside are local

Pass values in, return results out. Avoid global.

A list argument is shared

Change it in place and return None, or return a new one. Not both.

Before next time

- Finish sheet 08. Solutions go up after we discuss them.
- Take one program from an earlier sheet and split it into functions with a main.
- Read section 2.1 of Sedgewick, Wayne and Dondero, on defining functions.
- Bring the laptop.

Next: Lecture 09 — modules and files. Your functions in a file of their own, imported like math. And data that outlives the program.